RECONHECIMENTO AUTOMÁTICO DE EMOÇÕES

Baseado em Redes Neurais Convolucionais



Ana Pessoa 2023112690 · David Matos 2023143725

Instituto Superior de Engenharia de Coimbra – DEIS

Outubro 2025

Descrição do Caso de Estudo e Objetivos

Projeto

Classificação de expressões faciais humanas em 5 classes: **raiva, medo, alegria, tristeza e surpresa**



Desenvolvimento de um modelo CNN capaz de classificar corretamente a emoção em cada imagem facial.

Aplicações

- Interação Humano-Computador
- Análise Comportamental
- Interfaces Inteligentes
- Reconhecimento em Tempo Real

Objetivos do Projeto

Objetivos Específicos

Desenvolvimento do Modelo CNN

Desenvolver um modelo de rede neural convolutiva (CNN) capaz de classificar emoções faciais a partir de imagens.

Testes com Conjuntos de Dados Distintos

Testar o modelo utilizando dois conjuntos de dados: BHE (dados sintéticos) e FER2013 (imagens faciais reais).

Análise de Parâmetros Críticos

Analisar o impacto da resolução das imagens de entrada e do número de épocas de treino na performance do modelo.

Métricas de Avaliação

- Accuracy
- Recall
- F1-score
- AUC

Datasets Utilizados

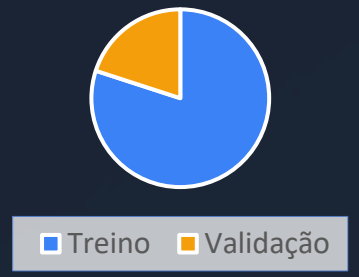
Basic Human Emotions (BHE)

Características

- Imagens geradas por inteligência artificial
- Controlo das condições, havendo uma iluminação uniforme
- Classes de emoções equilibradas

Utilização

- 80% para treino
- 20% para validação



FER2013

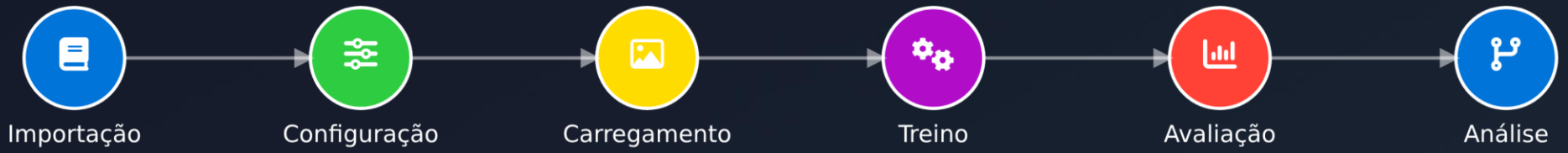
Características

- Imagens reais de rostos humanos
- Usado para avaliar generalização

Importante

O dataset FER2013 foi utilizado apenas para teste final, permitindo medir a capacidade de generalização do modelo a imagens reais.

Estrutura do Projeto



1. Importação de Bibliotecas

TensorFlow, Scikit-learn, Matplotlib e Seaborn para construção do modelo, análise de dados e visualização.

2. Configuração de Diretórios e Parâmetros

Definição de diretórios dos datasets e configuração de parâmetros como resolução de imagens e épocas.

3. Pré-processamento das imagens

Imagens carregadas e normalizadas para garantir valores de pixels consistentes.

4. Definição e treino do modelo

Modelo CNN treinado no dataset BHE com validação contínua.

5. Avaliação e Geração de Métricas

Avaliação de performance no dataset FER2013 para medir capacidade de generalização.

6. Análise de Métricas

Análise das métricas: accuracy, recall, F1-score e AUC.

Implementação de algorítmos

1. Importação de Bibliotecas

```python
# Função que treina o modelo (está no ficheiro train_model.py)
from train_model import train_model

# Métricas do scikit-learn para avaliar os resultados do modelo
# - classification_report -> mostra precisão, recall e f1-score
# - roc_auc_score -> mede a capacidade de distinguir classes (AUC)
# - confusion_matrix -> mostra acertos e erros por classe
from sklearn.metrics import classification_report, roc_auc_score, confusion_matrix

# label_binarize converte os rótulos em formato binário (necessário para AUC)
from sklearn.preprocessing import label_binarize

# Ferramentas do Keras para ler imagens organizadas por pastas
# (normaliza os píxeis e cria lotes automaticamente)
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Para carregar modelos previamente treinados (.h5)
from tensorflow.keras.models import load_model

# Bibliotecas de suporte
# - NumPy: operações matemáticas
# - Matplotlib e Seaborn: geração de gráficos
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Pathlib ajuda a criar caminhos de ficheiros/pastas de forma independente do sistema operativo
from pathlib import Path
```

2. Configuração de Diretórios e Parâmetros

```python
# ============================================================
# 1) Definir as pastas principais do dataset
# ------------------------------------------------------------
# ROOT        -> pasta principal do projeto (TP)
# DATASET_DIR -> onde estão as imagens
# TRAIN, VAL  -> subpastas com as imagens de treino e validação
# POOL        -> pasta temporária onde as imagens são reunidas
# ============================================================
ROOT        = Path(__file__).resolve().parent.parent        # .../TP
DATASET_DIR = ROOT / "dataset"
TRAIN = DATASET_DIR / "train"
VAL   = DATASET_DIR / "validation"
POOL  = DATASET_DIR / "_pool_tmp"   # pasta temporária (é apagada no final)
```

```python
# Cria as pastas das classes em train e validation se ainda não existirem
for c in CLASSES_OK:
    (TRAIN / c).mkdir(parents=True, exist_ok=True)
    (VAL / c).mkdir(parents=True, exist_ok=True)

# Cria também a pasta temporária "_pool_tmp"
POOL.mkdir(parents=True, exist_ok=True)
```

3. Pré-processamento das imagens

```python
# ============================================================
# 1. Preparação dos dados
# ------------------------------------------------------------
# As imagens são carregadas com o ImageDataGenerator, que:
# - Lê automaticamente as imagens em cada subpasta (classe)
# - Converte os valores dos píxeis para o intervalo [0, 1]
# ============================================================

train_datagen = ImageDataGenerator(rescale=1./255)   # Normaliza as imagens de treino
val_datagen   = ImageDataGenerator(rescale=1./255)   # Normaliza as imagens de validação

# Criação dos "geradores" que leem as imagens em lotes (batches)
train_generator = train_datagen.flow_from_directory(
    train_dir,                    # Caminho das imagens de treino
    target_size=(img_size, img_size),   # Redimensiona as imagens (64x64 ou 32x32)
    batch_size=batch_size,              # Número de imagens por lote
    class_mode='categorical',           # As classes são categóricas (emoções)
    shuffle=True                        # Baralha as imagens a cada época
)
print(f"Imagens de treino carregadas: {train_generator.samples} imagens")

# O gerador de validação lê as imagens sem baralhar
val_generator = val_datagen.flow_from_directory(
    val_dir,
    target_size=(img_size, img_size),
    batch_size=batch_size,
    class_mode='categorical',
    shuffle=False
)
```

Implementação de algorítmos

4. Definição e treino do modelo

```
# ============================================================
# --- Modelo mais avançado  ---
# Tem 5 blocos de convolução + pooling para aprender padrões
# cada vez mais complexos nas expressões faciais.
# ============================================================
model = Sequential()
model.add(Input(shape=input_shape))  # define tamanho das imagens de entrada

# 1º bloco: aprende padrões básicos (bordas, formas simples)
model.add(Conv2D(32, (3,3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(MaxPooling2D((2,2)))

# 2º bloco: combina padrões do anterior (ex: olhos, boca)
model.add(Conv2D(64, (3,3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(MaxPooling2D((2,2)))

# 3º bloco: aprende detalhes mais abstratos
model.add(Conv2D(128, (3,3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(MaxPooling2D((2,2)))

# 4º bloco: combina padrões complexos (expressões completas)
model.add(Conv2D(256, (3,3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(MaxPooling2D((2,2)))

# 5º bloco: reforça e generaliza as features aprendidas
model.add(Conv2D(256, (3,3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(MaxPooling2D((2,2)))
```

5. Avaliação e Geração de Métricas

```
# ============================================================
# 2.2 Relatório de Classificação (Precision, Recall, F1-score)
# ------------------------------------------------------------
# Esta métrica mostra o desempenho do modelo para cada emoção.
# Também inclui médias macro e ponderadas (importante para o relatório).
# ============================================================
report = classification_report(
    y_true, y_pred, target_names=class_labels, digits=4
)
print("\n--- Classification report ---")
print(report)

# Guarda o relatório num ficheiro de texto (.txt)
with open(exp_dir / "classification_report.txt", "w", encoding="utf-8") as f:
    f.write(report)

# ============================================================
# 2.3 Cálculo da AUC (Área sob a curva ROC)
# ------------------------------------------------------------
# Mede a capacidade do modelo distinguir cada emoção das restantes.
# A AUC macro é a média global (valor máximo = 1 → ótimo).
# ============================================================
y_true_bin = label_binarize(y_true, classes=list(range(num_classes)))

# Calcula a AUC individual de cada classe
auc_per_class = {
    class_labels[i]: roc_auc_score(y_true_bin[:, i], preds[:, i])
    for i in range(num_classes)
}

# Calcula a AUC média (macro)
auc_macro = roc_auc_score(
    y_true_bin, preds, average='macro', multi_class='ovr'
)

# Mostra os resultados no terminal
print("\nAUC por classe:")
for lbl, auc in auc_per_class.items():
    print(f"{lbl}: {auc:.4f}")
print(f"AUC macro (OvR): {auc_macro:.4f}")

# Guarda os valores de AUC num ficheiro de texto
with open(exp_dir / "auc.txt", "w", encoding="utf-8") as f:
    for lbl, auc in auc_per_class.items():
        f.write(f"{lbl}: {auc:.4f}\n")
    f.write(f"Macro: {auc_macro:.4f}\n")
```

```
# ============================================================
# 2.4 Matriz de Confusão + Sensibilidade/Especificidade
# ------------------------------------------------------------
# A matriz de confusão mostra onde o modelo acerta e erra.
# A sensibilidade (recall) e especificidade ajudam a perceber
# se o modelo reconhece bem cada emoção.
# ============================================================
cm = confusion_matrix(y_true, y_pred)

# Guarda os valores em ficheiro para análise posterior
with open(exp_dir / "sens_esp.txt", "w", encoding="utf-8") as f:
    f.write("--- Sensibilidade (Recall) e Especificidade ---\n")
    for i, lbl in enumerate(class_labels):
        TP = cm[i, i]  # verdadeiros positivos
        FN = cm[i, :].sum() - TP  # falsos negativos
        FP = cm[:, i].sum() - TP  # falsos positivos
        TN = cm.sum() - (TP + FN + FP)  # verdadeiros negativos

        # Cálculo das métricas
        sens = TP / (TP + FN) if (TP + FN) > 0 else 0.0
        esp  = TN / (TN + FP) if (TN + FP) > 0 else 0.0

        f.write(f"{lbl:>9s} | Sens: {sens:.4f} | Esp: {esp:.4f}\n")
```

Arquitetura da CNN

Rede Neural Convolucional

- Camada de Entrada — 64×64 pixels
- Camada Convolucional (Conv2D(32, 3x3, same)) — Batch Norm
- Camada de MaxPooling(2x2) — Redução de Dimensões
- Camada Convolucional (Conv2D(64, 3x3, same)) — Batch Norm
- Camada de MaxPooling(2x2) — Redução de Dimensões
- Camada Convolucional (Conv2D(128, 3x3, same)) — Batch Norm
- Camada de MaxPooling (2x2) — Redução de Dimensões
- Camada Convolucional (Conv2D(256, 3x3, same)) — Batch Norm
- Camada de MaxPooling (2x2) — Redução de Dimensões
- Camada de Achatamento — Vetor 1D
- Camada Densa — Dense(128, ReLU)
- Camada de Dropout — Dropout(0.25)
- Camada de Saída — Dense(num_classes=5, Softmax)
  Reconhecimento Automático de Emoções

Parâmetros do Treino

Função de Perda

Categorical Crossentropy

Adequada para classificação multi-classe

Otimizador

Adam

LR = 0,0036

Algoritmo eficiente para treinamento de redes neurais

Treino e Validação

Configuração do Treino

O treino foi realizado sobre o dataset **Basic Human Emotions (BHE)**.

Utilizou-se o **ImageDataGenerator** para **normalizar os pixels** entre 0 e 1 e garantir consistência entre amostras.

As imagens foram divididas **80% para treino e 20% para validação**, através do script fix_split_80_20.py.

O treino foi monitorizado a cada época, registando **accuracy e loss** de treino e validação.

Cada configuração (resolução × épocas) foi testada em experiências distintas.

A avaliação final foi feita com o dataset FER2013 (imagens reais).

Parâmetros de Treino

```
# ============================================================
# 3. Treino do modelo
# ------------------------------------------------------------
# O método fit() faz o treino propriamente dito.
# O modelo ajusta os pesos das suas camadas para reduzir o erro.
# Durante o treino, mostra a precisão e a perda a cada época.
# ============================================================

history = model.fit(
    train_generator,                    # Dados de treino
    steps_per_epoch=steps_per_epoch,    # Quantos lotes por época
    epochs=epochs,                      # Número de épocas (20 ou 30)
    validation_data=val_generator,      # Dados usados para validação
    validation_steps=validation_steps  # Quantos lotes usar na validação
)
```

Experiência 1 (64x64, 20 épocas)

Três experiências foram realizadas testando diferentes resoluções de imagem e números de épocas para avaliar seu impacto na performance do modelo CNN.

Experiência 1

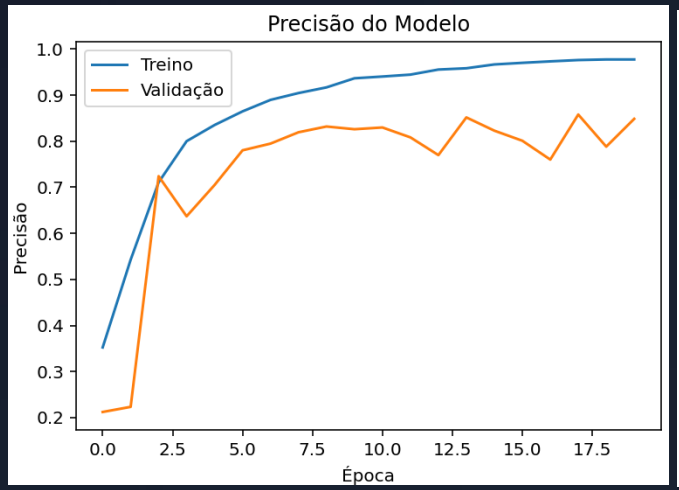
64×64 pixels, 20 épocas

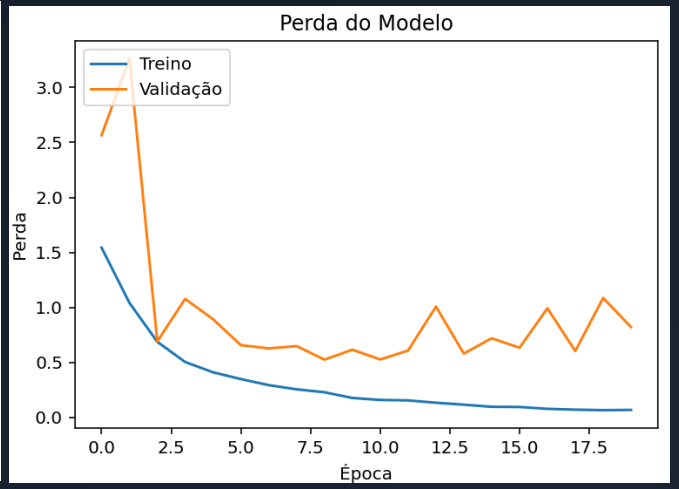
- Teste: **27.5%**
- AUC: **0.57**
- Accuracy treino: 97%
- Validação: 85%

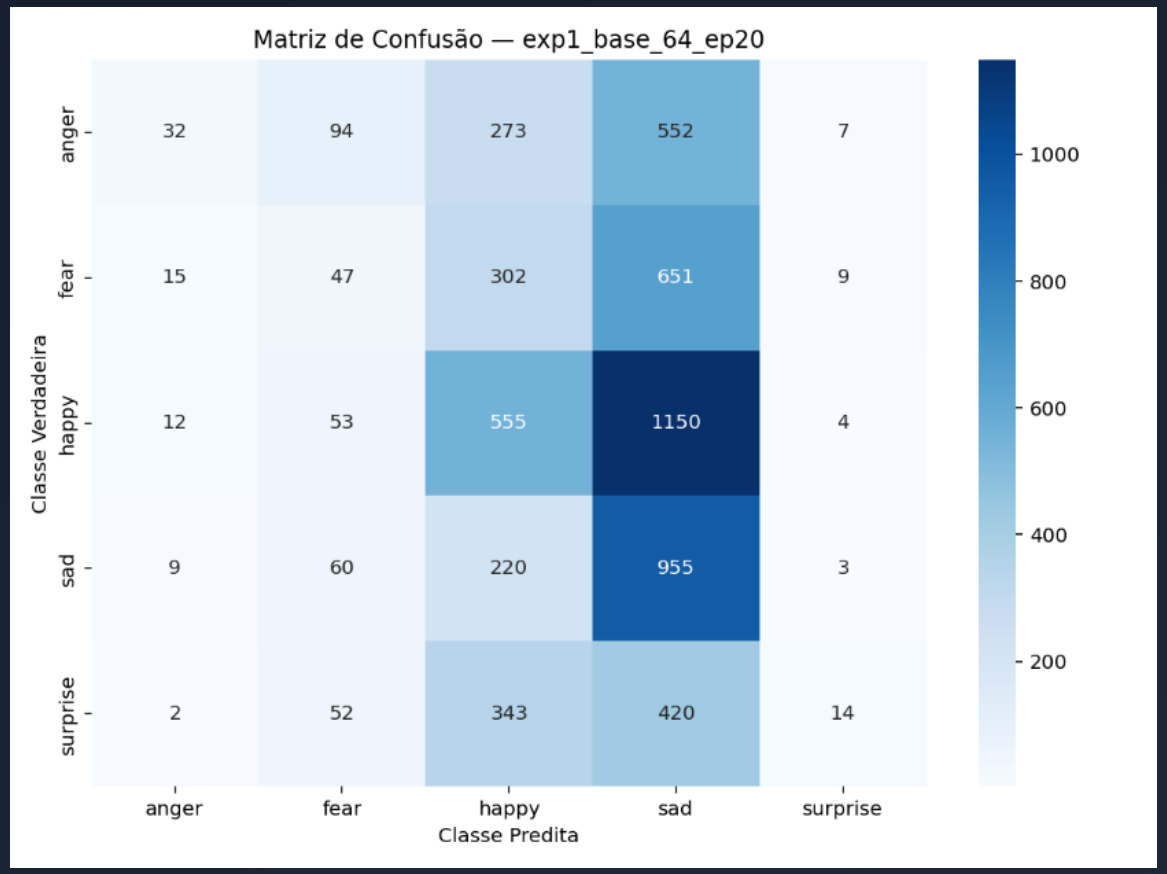
Melhor classe: Tristeza (recall 76%, F1 = 0.38)

Piores: Raiva, Medo e Surpresa (recall < 5%)

Intermédia: Alegria (*recall 31%, F1 = 0.32*)







Experiência 2 (64x64, 30 épocas)

Experiência 2

64×64 pixels, 30 épocas

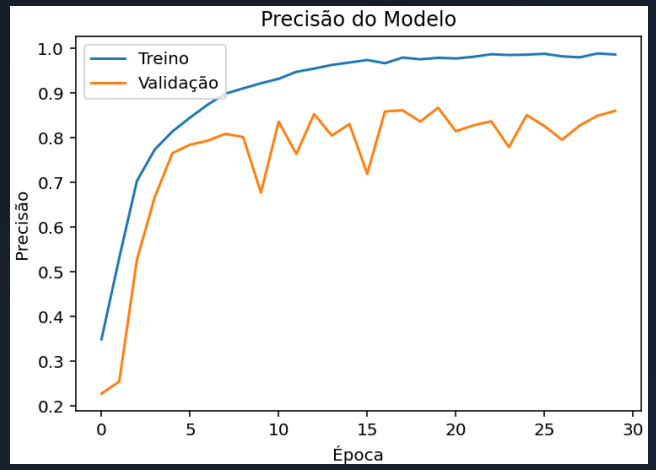
- Teste: **24.7%**
- AUC: **0.56**
- Accuracy treino: 99%
- Validação: 83%

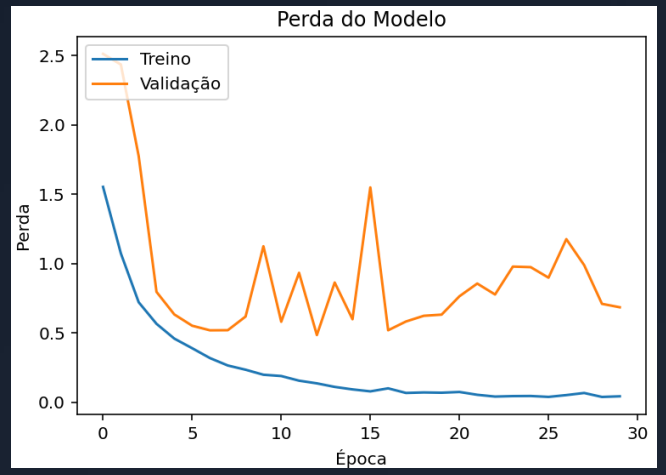
Melhor classe: Tristeza (recall 64%, F1 = 0.37)

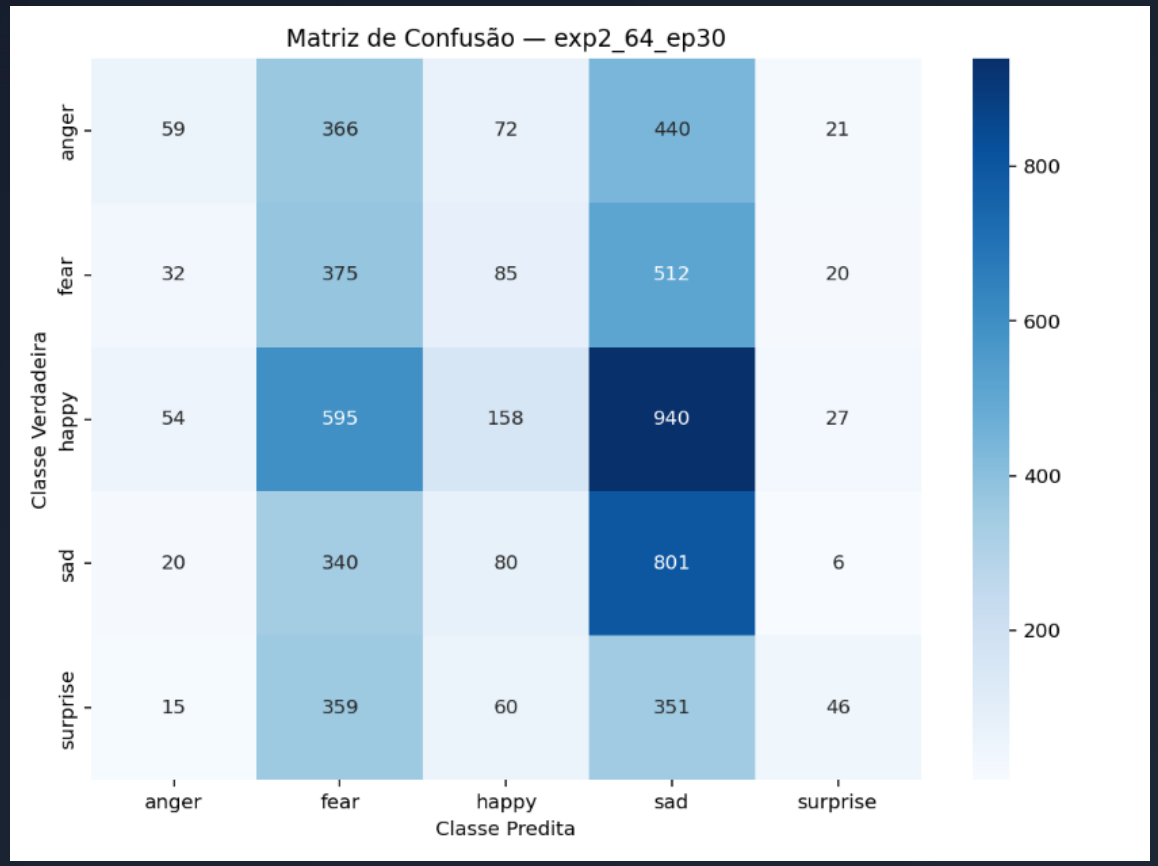
Piores: Raiva, Alegria e Surpresa (recall < 10%)

Intermédia: Medo (*recall 37%, F1 = 0.25*)

Overfitting observado







Experiência 3 (32x32, 20 épocas)

Experiência 3

32×32 pixels, 20 épocas

Teste: **22.4%**

AUC: **0.53**

Accuracy treino: 92%

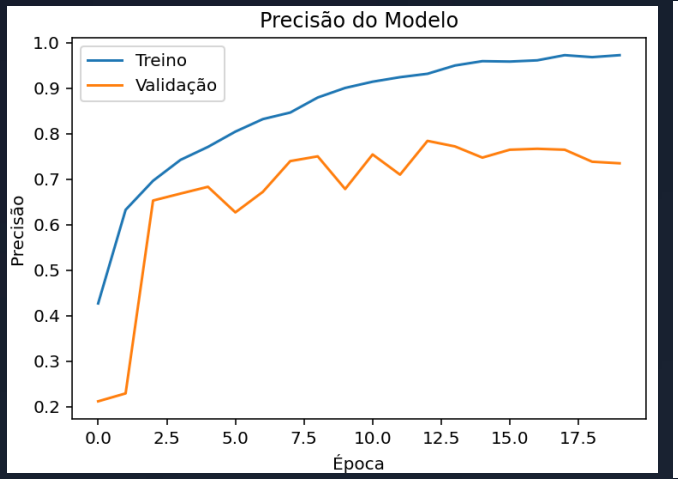
Validação: 78%

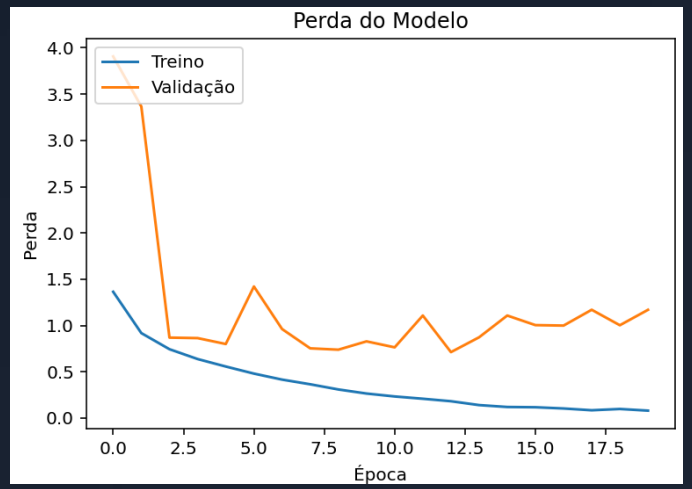
Perda de detalhes faciais devido à baixa resolução

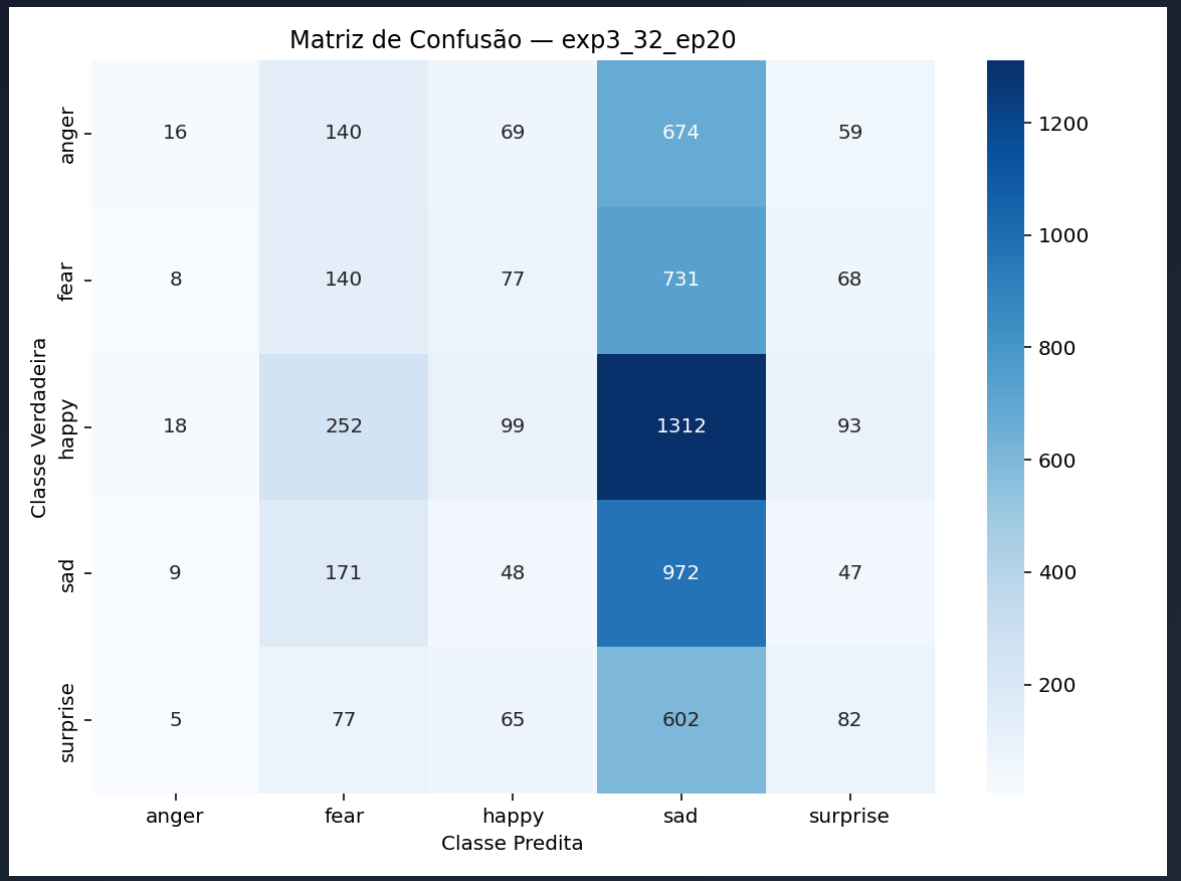
Melhor classe: Tristeza (recall 78%, F1 = 0.35)

Piores: Raiva, Medo, Alegria e Surpresa (recall < 15%)

Confusão: **emoções negativas mal distinguidas (fear ↔ sad)**

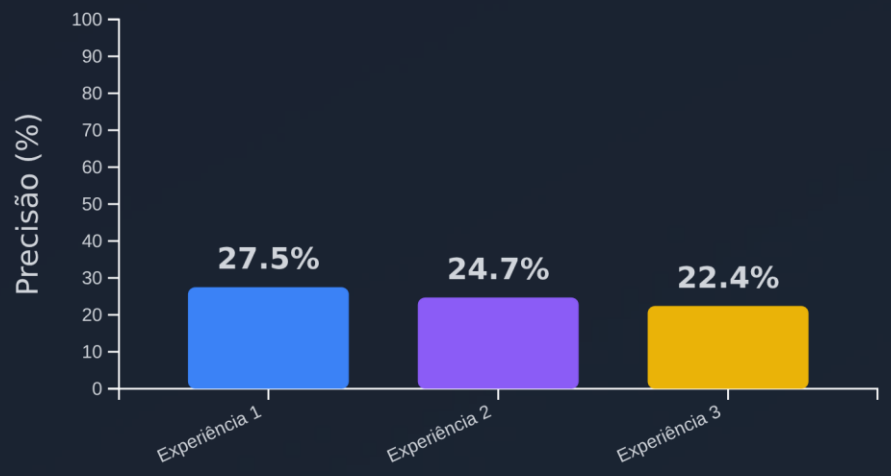






Comparação Geral e Conclusões

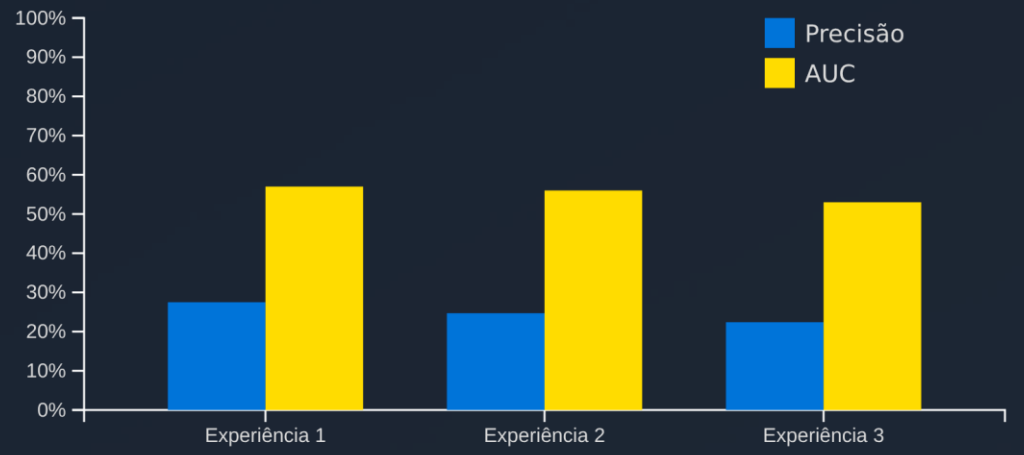
Análise dos Resultados



Observações Principais

- Aumentar épocas → overfitting (Experiência 1 vs. 2)
- Diminuir resolução → perda de detalhes (Experiência 1 vs. 3)
- Classe "tristeza" foi a melhor reconhecida em todas as experiências

Comparação das Experiências



Experiência	Resolução	Épocas	Precisão Teste	AUC Média
1	64×64	20	27.5%	0.57
2	64×64	30	24.7%	0.56
3	32×32	20	22.4%	0.53

Principais Conclusões

Principais Conclusões

Sobreapprentissage:
Mais épocas não significam melhor desempenho (overfitting observado)

Resolução das Imagens:
Diminuir resolução prejudica a capacidade de reconhecimento

Desafio dos Datasets:
Diferença significativa entre dados sintéticos e reais

Observação Final

O modelo CNN demonstrou boa capacidade de aprendizagem, mas dificuldade de generalização.

Estabelece uma base sólida para otimizações na Meta II (regularização, transfer learning).

Referências

Referências Utilizadas

Datasets

- Basic Human Emotions Dataset – Kaggle
- FER2013 – Kaggle

Bibliotecas

- TensorFlow
- Keras
- Scikit-learn
- Matplotlib
- Seaborn

Outras Referências

- ChatGPT
- Documentação TensorFlow

Recursos Específicos

- CNN architecture for facial emotion detection
- Overfitting in emotion recognition models